

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Escuela Profesional de Ingeniería de Software

Taller de Construcción de Software Móvil

INFORME TÉCNICO

Aplicación Móvil de Comercio Electrónico de Productos Naturales

NaturApp — Inspirada en Santa Natura

Aplicación de Conceptos: Arquitectura en Capas, Patrón MVVM, Persistencia Básica, Local (SQLite) y Remota (API REST)

Sesión 10 — Diseño de Software Móvil II

Contenido

1. Introducción y Objetivos
2. Conceptos de la Presentación Aplicados
3. Arquitectura del Proyecto NaturApp
4. Capa de Modelo (Model)
5. Capa de Datos — Persistencia Básica (AsyncStorage)
6. Capa de Datos — Persistencia Local (SQLite)
7. Capa de Datos — Persistencia Remota (API REST)
8. Capa de Lógica de Negocio (ViewModels / Custom Hooks)
9. Capa de Presentación (Screens y Components)
10. Navegación e Integración entre Capas
11. Resumen de Conceptos Aplicados

1. Introducción y Objetivos

El presente informe documenta el desarrollo de NaturApp, una aplicación móvil de comercio electrónico de productos naturales inspirada en Santa Natura (santanatura.com.pe). La app permite explorar un catálogo de productos naturales, gestionar un carrito de compras, realizar pedidos y administrar preferencias del usuario.

El objetivo principal es demostrar cómo cada concepto presentado en la Sesión 10 del curso — Diseño de Software Móvil II: Consolidación de Arquitectura, Persistencia e Integración — se aplica de forma práctica en una aplicación real desarrollada con React Native y Expo.

Conceptos cubiertos: Arquitectura en Capas, Patrón MVVM (adaptado como Model-View-ViewModel con Custom Hooks), Persistencia Básica (AsyncStorage), Persistencia Local Estructurada (SQLite), Persistencia Remota (API REST con fetch asíncrono), e Integración entre Capas.

2. Conceptos de la Presentación Aplicados

La siguiente tabla muestra el mapeo directo entre cada concepto de la presentación y su implementación en NaturApp:

Concepto de la Presentación	Implementación en NaturApp	Archivos Clave
Arquitectura en Capas (Presentación, Lógica, Datos)	Estructura de carpetas: screens/, viewmodels/, models/, services/	Toda la estructura del proyecto
Patrón MVVM (Model-View-ViewModel)	Models como interfaces, Screens como View, Custom Hooks como ViewModel	models/, viewmodels/, screens/
Persistencia Básica (SharedPreferences)	AsyncStorage para preferencias de usuario y sesión	services/storageService.js
Persistencia Local con SQLite	expo-sqlite para carrito y favoritos offline	services/databaseService.js
Persistencia Remota (APIs REST)	fetch() asíncrono al backend Express para productos y pedidos	services/apiService.js
Operaciones CRUD	Crear, leer, actualizar y eliminar productos del carrito y pedidos	databaseService.js, apiService.js
Asincronía (async/await)	Todas las operaciones de datos usan async/await sin bloquear la UI	Todos los services y viewmodels
Integración entre Capas	Flujo: Evento UI → ViewModel → Validación → Service → Resultado → Estado UI	viewmodels/ → services/
Separación de Responsabilidades	Cada archivo tiene una única responsabilidad clara	Todo el proyecto

3. Arquitectura del Proyecto NaturApp

NaturApp implementa una Arquitectura en Capas combinada con el patrón MVVM adaptado a React Native. La estructura de carpetas refleja la separación clara de responsabilidades:

Estructura de Directorios

```

NaturApp/
├── app/
│   ├── _layout.js           # Navegación (Expo Router)
│   ├── index.js            # Root Layout con Tabs
│   └── (tabs)/
│       ├── _layout.js      # Tab Navigator
│       ├── home.js         # Pantalla principal
│       ├── cart.js         # Carrito de compras
│       ├── orders.js       # Historial de pedidos
│       ├── profile.js      # Perfil y preferencias
│       └── product/[id].js  # Detalle de producto
├── src/
│   ├── models/             # CAPA MODELO
│   │   ├── Product.js
│   │   ├── CartItem.js
│   │   └── Order.js
│   ├── services/           # CAPA DE DATOS
│   │   ├── storageService.js # Persistencia Básica
│   │   ├── databaseService.js # Persistencia Local SQLite
│   │   └── apiService.js     # Persistencia Remota API
│   ├── viewmodels/         # CAPA LÓGICA (ViewModel)
│   │   ├── useProducts.js
│   │   ├── useCart.js
│   │   ├── useOrders.js
│   │   └── useProfile.js
│   └── components/         # Componentes reutilizables
│       ├── ProductCard.js
│       ├── CartItemRow.js
│       └── CategoryChip.js

```

✓ Concepto aplicado: Arquitectura en Capas

La estructura separa: Presentación (app/, components/) → Lógica de Negocio (viewmodels/) → Datos (services/). Cada capa tiene una responsabilidad única y se comunica solo con la capa adyacente, tal como describe la presentación.

✓ Concepto aplicado: Patrón MVVM adaptado a React Native

En React Native no existe ViewModel nativo. Se adapta usando Custom Hooks: Model = clases/objetos en models/, View = componentes en screens/ y components/, ViewModel = hooks en viewmodels/ que encapsulan estado y lógica de negocio.

4. Capa de Modelo (Model)

Los modelos representan las entidades del dominio del negocio. Definen la estructura de información que se persiste y transfiere entre capas.

4.1 Modelo Product

 `src/models/Product.js`

```
// Modelo que representa un producto natural del catálogo
// Equivale al 'Model' del patrón MVVM
export class Product {
  constructor({ id, name, description, price, image,
               category, stock, rating, benefits }) {
    this.id = id;
    this.name = name; // Nombre del producto
    this.description = description; // Descripción detallada
    this.price = price; // Precio en soles (PEN)
    this.image = image; // URL de la imagen
    this.category = category; // Categoría: 'superfoods', 'aceites', etc.
    this.stock = stock; // Cantidad disponible
    this.rating = rating || 0; // Calificación promedio
    this.benefits = benefits || []; // Lista de beneficios
  }

  // Factory method: crea instancia desde JSON de la API
  static fromJSON(json) {
    return new Product(json);
  }

  // Verifica si hay stock disponible
  isAvailable() {
    return this.stock > 0;
  }

  // Formato de precio para mostrar en la UI
  getFormattedPrice() {
    return `S/ ${this.price.toFixed(2)}`;
  }
}
```

4.2 Modelo CartItem

 `src/models/CartItem.js`

```
// Modelo que representa un item en el carrito de compras
export class CartItem {
  constructor({ id, productId, name, price, image,
               quantity }) {
    this.id = id; // ID único en SQLite
    this.productId = productId; // Referencia al producto
    this.name = name;
    this.price = price;
    this.image = image;
    this.quantity = quantity || 1;
  }

  // Calcula el subtotal de este item
}
```

```

getSubtotal() {
  return this.price * this.quantity;
}

// Crea instancia desde fila de SQLite
static fromRow(row) {
  return new CartItem({
    id: row.id,
    productId: row.product_id,
    name: row.name,
    price: row.price,
    image: row.image,
    quantity: row.quantity,
  });
}
}

```

4.3 Modelo Order

src/models/Order.js

```

// Modelo que representa un pedido completado
export class Order {
  constructor({ id, items, total, status, date,
    address }) {

    this.id = id;
    this.items = items || []; // Lista de CartItems
    this.total = total; // Monto total
    this.status = status || 'pendiente'; // Estado del pedido
    this.date = date || new Date().toISOString();
    this.address = address || '';
  }

  static fromJSON(json) {
    return new Order(json);
  }

  getFormattedDate() {
    return new Date(this.date).toLocaleDateString('es-PE');
  }

  getStatusColor() {
    const colors = {
      pendiente: '#F39C12',
      procesando: '#3498DB',
      enviado: '#8E44AD',
      entregado: '#27AE60',
    };
    return colors[this.status] || '#95A5A6';
  }
}

```

✓ Concepto aplicado: Model del patrón MVVM

Cada modelo encapsula los datos y la estructura de las entidades del sistema (Product, CartItem, Order). Los modelos incluyen métodos de utilidad (getFormattedPrice, getSubtotal, fromJSON) que pertenecen a la entidad, manteniendo la lógica de dominio separada de la presentación.

5. Capa de Datos — Persistencia Básica (AsyncStorage)

AsyncStorage es el equivalente en React Native de SharedPreferences (Android) o UserDefaults (iOS). Se usa para almacenar datos pequeños como configuraciones de usuario, tokens de sesión y preferencias simples.

 `src/services/storageService.js`

```
import AsyncStorage from
  '@react-native-async-storage/async-storage';

// =====
// PERSISTENCIA BÁSICA - AsyncStorage
// Equivale a SharedPreferences / UserDefaults
// Para datos pequeños: preferencias, tokens, config
// =====

const KEYS = {
  USER_NAME: '@naturapp_user_name',
  USER_EMAIL: '@naturapp_user_email',
  AUTH_TOKEN: '@naturapp_auth_token',
  THEME_DARK: '@naturapp_theme_dark',
  NOTIFICATIONS: '@naturapp_notifications',
  LAST_CATEGORY: '@naturapp_last_category',
  ONBOARDING_DONE: '@naturapp_onboarding',
};

const StorageService = {
  // --- GUARDAR datos (Create/Update) ---
  async saveUserProfile(name, email) {
    try {
      // multiSet guarda múltiples pares clave-valor
      await AsyncStorage.multiSet([
        [KEYS.USER_NAME, name],
        [KEYS.USER_EMAIL, email],
      ]);
      return true;
    } catch (error) {
      console.error('Error guardando perfil:', error);
      return false;
    }
  },

  // --- LEER datos (Read) ---
  async getUserProfile() {
    try {
      const values = await AsyncStorage.multiGet([
        KEYS.USER_NAME,
        KEYS.USER_EMAIL,
      ]);
      return {
        name: values[0][1] || '',
        email: values[1][1] || '',
      };
    } catch (error) {
      console.error('Error leyendo perfil:', error);
      return { name: '', email: '' };
    }
  },
};
```

```

// --- Preferencias booleanas ---
async setDarkTheme(enabled) {
  await AsyncStorage.setItem(
    KEYS.THEME_DARK, JSON.stringify(enabled)
  );
},

async isDarkTheme() {
  const val = await AsyncStorage.getItem(
    KEYS.THEME_DARK
  );
  return val ? JSON.parse(val) : false;
},

async setNotifications(enabled) {
  await AsyncStorage.setItem(
    KEYS.NOTIFICATIONS, JSON.stringify(enabled)
  );
},

async getNotifications() {
  const val = await AsyncStorage.getItem(
    KEYS.NOTIFICATIONS
  );
  return val ? JSON.parse(val) : true;
},

// --- Token de autenticación ---
async saveToken(token) {
  await AsyncStorage.setItem(KEYS.AUTH_TOKEN, token);
},

async getToken() {
  return await AsyncStorage.getItem(KEYS.AUTH_TOKEN);
},

// --- ELIMINAR datos (Delete) ---
async logout() {
  await AsyncStorage.multiRemove([
    KEYS.AUTH_TOKEN,
    KEYS.USER_NAME,
    KEYS.USER_EMAIL,
  ]);
},

// Guardar última categoría visitada
async saveLastCategory(category) {
  await AsyncStorage.setItem(
    KEYS.LAST_CATEGORY, category
  );
},

async getLastCategory() {
  return await AsyncStorage.getItem(
    KEYS.LAST_CATEGORY
  ) || 'todos';
},
};

export default StorageService;

```

✓ Concepto aplicado: Persistencia Básica (SharedPreferences / AsyncStorage)

AsyncStorage almacena pares clave-valor de forma persistente. Es ideal para configuraciones de usuario (tema oscuro, notificaciones), tokens de sesión y valores simples. Los datos sobreviven al cierre de la app. En la presentación se describe como el primer nivel de persistencia, para datos pequeños y preferencias.

6. Capa de Datos — Persistencia Local (SQLite)

SQLite es una base de datos relacional embebida en el dispositivo. Se usa para datos estructurados que requieren operaciones CRUD complejas: el carrito de compras y los productos favoritos.

 `src/services/databaseService.js`

```
import * as SQLite from 'expo-sqlite';

// =====
// PERSISTENCIA LOCAL ESTRUCTURADA - SQLite
// Base de datos relacional embebida
// Para carrito, favoritos, operaciones CRUD
// =====

let db = null;

const DatabaseService = {
  // --- INICIALIZAR base de datos ---
  async init() {
    db = await SQLite.openDatabaseAsync('naturapp.db');

    // Crear tabla del carrito de compras
    await db.execAsync(`
      CREATE TABLE IF NOT EXISTS cart (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        product_id TEXT NOT NULL UNIQUE,
        name TEXT NOT NULL,
        price REAL NOT NULL,
        image TEXT,
        quantity INTEGER DEFAULT 1
      );
    `);

    // Crear tabla de favoritos
    await db.execAsync(`
      CREATE TABLE IF NOT EXISTS favorites (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        product_id TEXT NOT NULL UNIQUE,
        name TEXT NOT NULL,
        price REAL NOT NULL,
        image TEXT,
        added_date TEXT DEFAULT
          (datetime('now'))
      );
    `);
  },

  // === OPERACIONES CRUD DEL CARRITO ===

  // CREATE: Agregar producto al carrito
  async addToCart(product) {
    const result = await db.runAsync(
      `INSERT OR REPLACE INTO cart
        (product_id, name, price, image, quantity)
        VALUES (?, ?, ?, ?,
          COALESCE(
            (SELECT quantity + 1 FROM cart
              WHERE product_id = ?), 1))`);
  },
};
```

```

        [product.id, product.name,
         product.price, product.image, product.id]
    );
    return result.lastInsertRowId;
},

// READ: Obtener todos los items del carrito
async getCartItems() {
    const rows = await db.getAllAsync(
        'SELECT * FROM cart ORDER BY id DESC'
    );
    return rows;
},

// UPDATE: Cambiar cantidad de un item
async updateCartQuantity(productId, quantity) {
    if (quantity <= 0) {
        return this.removeFromCart(productId);
    }
    await db.runAsync(
        'UPDATE cart SET quantity = ? WHERE product_id = ?',
        [quantity, productId]
    );
},

// DELETE: Eliminar un item del carrito
async removeFromCart(productId) {
    await db.runAsync(
        'DELETE FROM cart WHERE product_id = ?',
        [productId]
    );
},

// READ: Obtener total del carrito
async getCartTotal() {
    const result = await db.getFirstAsync(
        'SELECT SUM(price * quantity) as total FROM cart'
    );
    return result?.total || 0;
},

// DELETE: Vaciar el carrito completo
async clearCart() {
    await db.runAsync('DELETE FROM cart');
},

// READ: Contar items en el carrito
async getCartCount() {
    const result = await db.getFirstAsync(
        'SELECT SUM(quantity) as count FROM cart'
    );
    return result?.count || 0;
},

// === OPERACIONES CRUD DE FAVORITOS ===

async addFavorite(product) {
    await db.runAsync(
        `INSERT OR IGNORE INTO favorites
         (product_id, name, price, image)`
    );
}

```

```

        VALUES (?, ?, ?, ?)`\`,
        [product.id, product.name,
        product.price, product.image]
    );
},

async removeFavorite(productId) {
    await db.runAsync(
        'DELETE FROM favorites WHERE product_id = ?',
        [productId]
    );
},

async isFavorite(productId) {
    const row = await db.getFirstAsync(
        'SELECT id FROM favorites WHERE product_id = ?',
        [productId]
    );
    return !!row;
},

async getFavorites() {
    return await db.getAllAsync(
        'SELECT * FROM favorites ORDER BY added_date DESC'
    );
},
};

export default DatabaseService;

```

✓ Concepto aplicado: Persistencia Local Estructurada con SQLite

SQLite permite almacenar datos en tablas con relaciones y consultas SQL. Se implementan las 4 operaciones CRUD (Create, Read, Update, Delete) sobre el carrito y los favoritos. Los datos persisten offline y no requieren conexión a Internet. La presentación indica que SQLite es ideal para listas, inventarios, catálogos y formularios.

7. Capa de Datos — Persistencia Remota (API REST)

La persistencia remota conecta la app con un servidor backend que expone endpoints HTTP para operaciones CRUD. Las operaciones se ejecutan de forma asíncrona para no bloquear la interfaz.

 `src/services/apiService.js`

```
import StorageService from './storageService';

// =====
// PERSISTENCIA REMOTA - API REST
// Conexión con backend Express/Node.js
// Usa fetch con async/await (asincronía)
// =====

const BASE_URL = 'http://192.168.1.100:9090/api';

// Helper para peticiones HTTP con manejo de errores
async function request(endpoint, options = {}) {
  try {
    const token = await StorageService.getToken();
    const response = await fetch(
      `${BASE_URL}${endpoint}`,
      {
        ...options,
        headers: {
          'Content-Type': 'application/json',
          ...(token && {
            Authorization: `Bearer ${token}`
          }),
          ...options.headers,
        },
      },
    );
    if (!response.ok) {
      throw new Error(
        `HTTP ${response.status}: ${response.statusText}`
      );
    }
    return await response.json();
  } catch (error) {
    console.error(`API Error [${endpoint}]:`, error);
    throw error;
  }
}

const ApiService = {
  // === PRODUCTOS (READ) ===
  async getProducts(category = null) {
    const query = category
      ? `?category=${category}` : '';
    return await request(`/products${query}`);
  },

  async getProductById(id) {
    return await request(`/products/${id}`);
  },
}
```

```

async searchProducts(query) {
  return await request(
    `/products/search?q=${encodeURIComponent(query)}`
  );
},

// === PEDIDOS (CRUD completo) ===
// CREATE: Crear nuevo pedido
async createOrder(orderData) {
  return await request('/orders', {
    method: 'POST',
    body: JSON.stringify(orderData),
  });
},

// READ: Obtener historial de pedidos
async getOrders() {
  return await request('/orders');
},

// READ: Detalle de un pedido
async getOrderById(id) {
  return await request(`/orders/${id}`);
},

// === AUTENTICACIÓN ===
async login(email, password) {
  const data = await request('/auth/login', {
    method: 'POST',
    body: JSON.stringify({ email, password }),
  });
  // Guarda token en persistencia básica
  if (data.token) {
    await StorageService.saveToken(data.token);
    await StorageService.saveUserProfile(
      data.user.name, data.user.email
    );
  }
  return data;
},

// === CATEGORÍAS ===
async getCategories() {
  return await request('/categories');
},
};

export default ApiService;

```

✓ Concepto aplicado: Persistencia Remota y Asincronía

El servicio API usa `fetch()` con `async/await` para ejecutar operaciones HTTP en segundo plano sin congelar la interfaz. Esto implementa el concepto de asincronía que la presentación describe como fundamental: las operaciones remotas no deben bloquear la UI. Se conecta con el servidor Express (`server.js` del proyecto backend del usuario).

8. Capa de Lógica de Negocio (ViewModels)

Los ViewModels se implementan como Custom Hooks de React. Encapsulan el estado, la lógica de negocio, las validaciones y la coordinación entre servicios de datos. Cada ViewModel conecta la Vista con los Servicios de datos sin que la UI conozca los detalles de persistencia.

8.1 ViewModel de Productos

 `src/viewmodels/useProducts.js`

```
import { useState, useEffect, useCallback } from 'react';
import ApiService from '../services/apiService';
import StorageService from '../services/storageService';
import { Product } from '../models/Product';

// =====
// VIEWMODEL: Gestiona lógica de productos
// Conecta View (pantallas) con Data (API)
// =====
export function useProducts() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [category, setCategory] = useState('todos');
  const [searchQuery, setSearchQuery] = useState('');

  // Cargar productos desde la API (persistencia remota)
  const loadProducts = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const cat = category === 'todos'
        ? null : category;
      const data = await ApiService.getProducts(cat);
      // Convertir JSON a instancias del Modelo
      const mapped = data.map(p => Product.fromJSON(p));
      setProducts(mapped);
      // Guardar última categoría (persist. básica)
      await StorageService.saveLastCategory(category);
    } catch (err) {
      setError('No se pudieron cargar los productos');
    } finally {
      setLoading(false);
    }
  }, [category]);

  // Búsqueda de productos
  const search = useCallback(async (query) => {
    if (!query.trim()) {
      loadProducts();
      return;
    }
    setLoading(true);
    try {
      const data = await ApiService.searchProducts(query);
      setProducts(data.map(p => Product.fromJSON(p)));
    } catch (err) {
      setError('Error en la búsqueda');
    } finally {
      setLoading(false);
    }
  }, []);
}
```

```

    }
  }, []);

  // Restaurar última categoría al iniciar
  useEffect(() => {
    StorageService.getLastCategory()
      .then(cat => setCategory(cat));
  }, []);

  // Recargar cuando cambia la categoría
  useEffect(() => { loadProducts(); }, [loadProducts]);

  return {
    products, loading, error,
    category, setCategory,
    searchQuery, setSearchQuery,
    search, refresh: loadProducts,
  };
}

```

8.2 ViewModel del Carrito

 **src/viewmodels/useCart.js**

```

import { useState, useEffect, useCallback } from 'react';
import DatabaseService from '../services/databaseService';
import ApiService from '../services/apiService';
import { CartItem } from '../models/CartItem';

// =====
// VIEWMODEL: Gestiona lógica del carrito
// Conecta View con SQLite (local) y API (remoto)
// =====
export function useCart() {
  const [items, setItems] = useState([]);
  const [total, setTotal] = useState(0);
  const [count, setCount] = useState(0);
  const [loading, setLoading] = useState(false);

  const loadCart = useCallback(async () => {
    setLoading(true);
    try {
      const rows = await DatabaseService.getCartItems();
      setItems(rows.map(r => CartItem.fromRow(r)));
      const t = await DatabaseService.getCartTotal();
      setTotal(t);
      const c = await DatabaseService.getCartCount();
      setCount(c);
    } finally {
      setLoading(false);
    }
  }, []);

  // AGREGAR producto (validación + persistencia)
  const addItem = useCallback(async (product) => {
    // VALIDACIÓN: Lógica de negocio
    if (!product.isAvailable()) {
      throw new Error('Producto sin stock');
    }
  }, []);
}

```

```

    }
    await DatabaseService.addToCart(product);
    await loadCart(); // Actualizar estado
  }, [loadCart]);

// ACTUALIZAR cantidad
const updateQuantity = useCallback(
  async (productId, qty) => {
    // VALIDACIÓN: no permitir negativos
    if (qty < 0) return;
    await DatabaseService.updateCartQuantity(
      productId, qty
    );
    await loadCart();
  }, [loadCart]
);

// ELIMINAR item
const removeItem = useCallback(
  async (productId) => {
    await DatabaseService.removeFromCart(productId);
    await loadCart();
  }, [loadCart]
);

// CHECKOUT: Envía pedido a la API remota
const checkout = useCallback(
  async (address) => {
    // VALIDACIÓN
    if (items.length === 0) {
      throw new Error('El carrito está vacío');
    }
    if (!address.trim()) {
      throw new Error('Ingrese una dirección');
    }
    // Enviar a API remota (persistencia remota)
    const order = await ApiService.createOrder({
      items: items.map(i => ({
        productId: i.productId,
        name: i.name,
        price: i.price,
        quantity: i.quantity,
      })),
      total,
      address,
    });
    // Limpiar carrito local (SQLite)
    await DatabaseService.clearCart();
    await loadCart();
    return order;
  }, [items, total, loadCart]
);

useEffect(() => {
  DatabaseService.init().then(loadCart);
}, [loadCart]);

return {
  items, total, count, loading,
  addItem, updateQuantity, removeItem,
  checkout, refresh: loadCart,

```

```
};
}
```

✓ Concepto aplicado: ViewModel del patrón MVVM + Integración entre Capas

Los ViewModels (Custom Hooks) implementan el flujo de integración descrito en la presentación: (1) Evento del usuario → (2) ViewModel recibe la acción → (3) Validación de datos → (4) Operación de datos (SQLite o API) → (5) Resultado → (6) Actualización del estado UI. La vista NUNCA accede directamente a la base de datos.

8.3 ViewModel de Pedidos

 `src/viewmodels/useOrders.js`

```
import { useState, useCallback } from 'react';
import ApiService from '../services/apiService';
import { Order } from '../models/Order';

export function useOrders() {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const loadOrders = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await ApiService.getOrders();
      setOrders(data.map(o => Order.fromJSON(o)));
    } catch (err) {
      setError('No se pudo cargar el historial');
    } finally {
      setLoading(false);
    }
  }, []);

  return { orders, loading, error,
    refresh: loadOrders };
}
```

8.4 ViewModel del Perfil

 `src/viewmodels/useProfile.js`

```
import { useState, useEffect, useCallback } from 'react';
import StorageService from '../services/storageService';

export function useProfile() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [darkTheme, setDarkTheme] = useState(false);
  const [notifications, setNotifications] =
    useState(true);
```

```
const loadProfile = useCallback(async () => {
  const profile = await StorageService.getUserProfile();
  setName(profile.name);
  setEmail(profile.email);
  setDarkTheme(await StorageService.isDarkTheme());
  setNotifications(
    await StorageService.getNotifications()
  );
}, []);

const saveProfile = useCallback(async () => {
  await StorageService.saveUserProfile(name, email);
}, [name, email]);

const toggleTheme = useCallback(async () => {
  const newVal = !darkTheme;
  setDarkTheme(newVal);
  await StorageService.setDarkTheme(newVal);
}, [darkTheme]);

const toggleNotifications = useCallback(async () => {
  const newVal = !notifications;
  setNotifications(newVal);
  await StorageService.setNotifications(newVal);
}, [notifications]);

useEffect(() => { loadProfile(); }, [loadProfile]);

return {
  name, setName, email, setEmail,
  darkTheme, notifications,
  saveProfile, toggleTheme,
  toggleNotifications,
};
}
```

9. Capa de Presentación (Screens y Components)

La capa de presentación contiene las pantallas (Views) y componentes reutilizables. Cada pantalla usa un ViewModel para obtener datos y ejecutar acciones, sin conocer los detalles de persistencia.

9.1 Pantalla Principal (Home)

 `app/(tabs)/home.js`

```
import React from 'react';
import {
  View, Text, FlatList, TextInput,
  StyleSheet, ActivityIndicator,
  ScrollView, RefreshControl,
} from 'react-native';
import { useProducts } from
  '../../../src/viewmodels/useProducts';
import { useCart } from
  '../../../src/viewmodels/useCart';
import ProductCard from
  '../../../src/components/ProductCard';
import CategoryChip from
  '../../../src/components/CategoryChip';

const CATEGORIES = [
  'todos', 'superfoods', 'aceites',
  'capsulas', 'infusiones', 'miel',
];

export default function HomeScreen() {
  // Obtener datos del ViewModel (NO del servicio)
  const {
    products, loading, error,
    category, setCategory,
    searchQuery, setSearchQuery,
    search, refresh,
  } = useProducts();
  const { addItem } = useCart();

  const handleAddToCart = async (product) => {
    try {
      await addItem(product);
      alert(`${product.name} agregado al carrito`);
    } catch (e) {
      alert(e.message);
    }
  };

  return (
    <View style={styles.container}>
      {/* Barra de búsqueda */}
      <TextInput
        style={styles.searchBar}
        placeholder='Buscar productos naturales...'
        value={searchQuery}
        onChangeText={setSearchQuery}
        onSubmitEditing={() => search(searchQuery)}
      />
```

```

    {/* Chips de categorías */}
    <ScrollView horizontal
      showsHorizontalScrollIndicator={false}
      style={styles.categories}>
      {CATEGORIES.map(cat => (
        <CategoryChip
          key={cat}
          label={cat}
          active={category === cat}
          onPress={() => setCategory(cat)}
        />
      ))}
    </ScrollView>

    {/* Lista de productos */}
    {loading ? (
      <ActivityIndicator size='large'
        color='#148F77' />
    ) : error ? (
      <Text style={styles.error}>{error}</Text>
    ) : (
      <FlatList
        data={products}
        numColumns={2}
        keyExtractor={item => item.id}
        renderItem={({ item }) => (
          <ProductCard
            product={item}
            onAddToCart={() =>
              handleAddToCart(item)}
          />
        )}
        refreshControl={
          <RefreshControl
            refreshing={loading}
            onRefresh={refresh}
          />
        }
      />
    )}
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F5F5F5',
    padding: 12 },
  searchBar: { backgroundColor: 'FFF',
    borderRadius: 10, padding: 12,
    fontSize: 15, marginBottom: 10,
    borderWidth: 1, borderColor: '#E0E0E0' },
  categories: { marginBottom: 10,
    maxHeight: 44 },
  error: { color: '#E74C3C', textAlign: 'center',
    marginTop: 40, fontSize: 16 },
});

```

✓ **Concepto aplicado: View del patrón MVVM + Capa de Presentación**

La pantalla Home es la View que se comunica SOLO con los ViewModels (useProducts, useCart). No importa ningún servicio de datos directamente. Captura eventos del usuario (búsqueda, selección de categoría, agregar al carrito) y los delega al ViewModel para su procesamiento.

9.2 Pantalla del Carrito

 `app/(tabs)/cart.js`

```
import React, { useState } from 'react';
import {
  View, Text, FlatList, TextInput,
  TouchableOpacity, StyleSheet, Alert,
} from 'react-native';
import { useCart } from
  '../src/viewmodels/useCart';
import CartItemRow from
  '../src/components/CartItemRow';

export default function CartScreen() {
  const {
    items, total, loading,
    updateQuantity, removeItem, checkout,
  } = useCart();
  const [address, setAddress] = useState('');

  const handleCheckout = async () => {
    try {
      const order = await checkout(address);
      Alert.alert('Pedido Creado',
        `Pedido #${order.id} registrado.`);
      setAddress('');
    } catch (e) {
      Alert.alert('Error', e.message);
    }
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>
        Mi Carrito ({items.length} items)
      </Text>

      <FlatList
        data={items}
        keyExtractor={item =>
          item.productId.toString()}
        renderItem={({ item }) => (
          <CartItemRow
            item={item}
            onIncrease={() =>
              updateQuantity(
                item.productId,
                item.quantity + 1)}
            onDecrease={() =>
              updateQuantity(
                item.productId,
                item.quantity - 1)}
            onRemove={() =>
```



```

        removeItem(item.productId)
      />
    })
    ListEmptyComponent={
      <Text style={styles.empty}>
        Tu carrito está vacío
      </Text>
    }
  />

  {items.length > 0 && (
    <View style={styles.footer}>
      <TextInput
        style={styles.addressInput}
        placeholder='Dirección de entrega'
        value={address}
        onChangeText={setAddress}
      />
      <View style={styles.totalRow}>
        <Text style={styles.totalLabel}>
          Total:</Text>
        <Text style={styles.totalValue}>
          S/ {total.toFixed(2)}</Text>
      </View>
      <TouchableOpacity
        style={styles.checkoutBtn}
        onPress={handleCheckout}>
        <Text style={styles.checkoutText}>
          Realizar Pedido</Text>
      </TouchableOpacity>
    </View>
  )}
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F5F5F5',
    padding: 16 },
  title: { fontSize: 22, fontWeight: 'bold',
    color: '#1A5276', marginBottom: 16 },
  empty: { textAlign: 'center', marginTop: 60,
    fontSize: 16, color: '#999' },
  footer: { borderTopWidth: 1,
    borderTopColor: '#E0E0E0', paddingTop: 16 },
  addressInput: { backgroundColor: '#FFF',
    borderRadius: 8, padding: 12,
    borderWidth: 1, borderColor: '#DDD',
    marginBottom: 12 },
  totalRow: { flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 12 },
  totalLabel: { fontSize: 18, fontWeight: '600',
    color: '#333' },
  totalValue: { fontSize: 20, fontWeight: 'bold',
    color: '#148F77' },
  checkoutBtn: { backgroundColor: '#148F77',
    borderRadius: 10, padding: 16,
    alignItems: 'center' },
  checkoutText: { color: '#FFF', fontSize: 16,
    fontWeight: 'bold' },
});

```

9.3 Componentes Reutilizables

 **src/components/ProductCard.js**

```
import React from 'react';
import { View, Text, Image,
  TouchableOpacity, StyleSheet } from 'react-native';
import { Link } from 'expo-router';

export default function ProductCard({
  product, onAddToCart }) {
  return (
    <View style={styles.card}>
      <Link href={`\product/${product.id}`}>
        <Image
          source={{ uri: product.image }}
          style={styles.image}
        />
      </Link>
      <Text style={styles.name} numberOfLines={2}>
        {product.name}
      </Text>
      <Text style={styles.category}>
        {product.category}
      </Text>
      <View style={styles.row}>
        <Text style={styles.price}>
          {product.getFormattedPrice()}
        </Text>
        <TouchableOpacity
          style={styles.addBtn}
          onPress={onAddToCart}>
          <Text style={styles.addText}>+</Text>
        </TouchableOpacity>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: { flex: 1, backgroundColor: '#FFF',
    borderRadius: 12, margin: 6, padding: 10,
    elevation: 2, maxWidth: '48%' },
  image: { width: '100%', height: 120,
    borderRadius: 8 },
  name: { fontSize: 14, fontWeight: '600',
    color: '#333', marginTop: 8 },
  category: { fontSize: 11, color: '#888',
    textTransform: 'capitalize', marginTop: 2 },
  row: { flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center', marginTop: 8 },
  price: { fontSize: 16, fontWeight: 'bold',
    color: '#148F77' },
  addBtn: { backgroundColor: '#148F77',
    borderRadius: 16, width: 32, height: 32,
    alignItems: 'center',
    justifyContent: 'center' },
  addText: { color: '#FFF', fontSize: 20,
    fontWeight: 'bold' },
});
```

```
});
```

10. Navegación e Integración entre Capas

La navegación usa Expo Router con file-based routing. El root layout configura la navegación por tabs e inicializa la base de datos SQLite al arrancar la app.

app/_layout.js

```
import { useEffect } from 'react';
import { Stack } from 'expo-router';
import DatabaseService from
  '../src/services/databaseService';

export default function RootLayout() {
  // Inicializar SQLite al arrancar la app
  useEffect(() => {
    DatabaseService.init()
      .then(() => console.log('DB lista'))
      .catch(err => console.error(
        'Error DB:', err));
  }, []);

  return (
    <Stack screenOptions={{
      headerStyle: {
        backgroundColor: '#1A5276' },
        headerTintColor: '#FFF',
      }}>
      <Stack.Screen name='(tabs)'
        options={{ headerShown: false }} />
      <Stack.Screen name='product/[id]'
        options={{ title: 'Detalle' }} />
    </Stack>
  );
}
```

app/(tabs)/_layout.js

```
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

export default function TabLayout() {
  return (
    <Tabs screenOptions={{
      tabBarActiveTintColor: '#148F77',
      headerStyle: {
        backgroundColor: '#1A5276' },
      headerTintColor: '#FFF',
    }}>
      <Tabs.Screen name='home' options={{
        title: 'NaturApp',
        tabBarIcon: ({ color, size }) =>
          <Ionicons name='leaf' size={size}
            color={color} /> }} />
      <Tabs.Screen name='cart' options={{
        title: 'Carrito',
        tabBarIcon: ({ color, size }) =>
          <Ionicons name='cart' size={size}
            color={color} /> }} />
      <Tabs.Screen name='orders' options={{
```

```
        title: 'Pedidos',
        tabBarIcon: ({ color, size }) =>
          <Icons name='receipt' size={size}
            color={color} /> {} />
      <Tabs.Screen name='profile' options={{
        title: 'Perfil',
        tabBarIcon: ({ color, size }) =>
          <Icons name='person' size={size}
            color={color} /> {} />
      }} />
    </Tabs>
  );
}
```

✓ Concepto aplicado: Integración entre Capas del Sistema

El flujo completo de integración sigue los 6 pasos descritos en la presentación: (1) El usuario toca 'Agregar al carrito' → (2) HomeScreen llama addItem del ViewModel useCart → (3) useCart valida stock disponible → (4) DatabaseService inserta en SQLite → (5) SQLite retorna el resultado → (6) useCart actualiza el estado y la UI se re-renderiza automáticamente. La vista NUNCA accede directamente a la base de datos.

11. Resumen de Conceptos Aplicados

La siguiente tabla resume cómo cada concepto de la Sesión 10 se aplica de forma concreta en el código de NaturApp:

Concepto	Archivo(s)	Líneas Clave
Arquitectura en Capas	Toda la estructura	models/ → services/ → viewmodels/ → screens/
MVVM - Model	models/Product.js, CartItem.js, Order.js	Clases con métodos de dominio
MVVM - View	app/(tabs)/*.js, components/*.js	Solo renderiza UI y delega a ViewModels
MVVM - ViewModel	viewmodels/use*.js	Custom Hooks con estado y lógica
Persistencia Básica	services/storageService.js	AsyncStorage.setItem/getItem
Persistencia Local (SQLite)	services/databaseService.js	db.runAsync, db.getAllAsync (CRUD)
Persistencia Remota	services/apiService.js	fetch() con async/await a la API REST
Operaciones CRUD	databaseService.js + apiService.js	INSERT, SELECT, UPDATE, DELETE
Asincronía (async/await)	Todos los services y viewmodels	async/await en cada operación de I/O
Validación en Lógica	viewmodels/useCart.js	Verifica stock, carrito vacío, dirección
Integración entre Capas	UI → ViewModel → Service	6 pasos: Evento → VM → Validación → Data → Resultado → UI

NaturApp demuestra que todos los conceptos presentados en la Sesión 10 se aplican de forma práctica y coherente en una aplicación real. La consolidación de arquitectura, la separación de responsabilidades, el patrón MVVM, y los tres niveles de persistencia trabajan en conjunto para crear un sistema robusto, mantenible y escalable.